Part 1

What is DevOps ?

DevOps is a set of practices that combines **software development** and **IT operations**. Its primary goal is to **automate** and improve the flow of the entire software development life cycle, from initial development to testing and final deployment.

Key benefits of DevOps include:

- **Faster builds**
- **Safer deployments**
- **Easier maintainability**

By implementing these practices, teams can avoid issues that arise from manual, uncoordinated processes—such as breaking applications in production—which are particularly critical for high-stakes, user-facing (P0/P1) applications.

- (Development + Operations = DevOps)

Lifecycle of DevOps - 7 Pillars

- **Plan:** Defining the requirements, roadmap, and project scope with the product and engineering teams before coding starts.
  ↓
- **Develop:** The actual writing of the source code and setting up the repository.
  ↓
- **Build:** Using continuous integration (CI) tools to automatically compile and package the code.
  ↓
- **Test:** Executing automated tests to ensure the software functions correctly and meets quality standards before release.
  ↓
- **Release:** Finalizing the build and preparing the application to move into production environments
  ↓

- **Deploy:** Using continuous deployment (CD) to push the application to servers where users can access it.
  ↓
- **Monitor:** Tracking application performance and capturing user data in real-time to identify and resolve issues.

Linux in DevOps

Linux commands are essential in DevOps because servers, containers, and CI/CD pipelines primarily run on Linux environments.

- **Navigation & Listing**
  - `pwd` → Shows the current working directory path
  - `ls` → Lists files and directories in the current folder
  - `ls -l` → Lists files with detailed info (permissions, size, owner, etc.)
  - `ls -a` → Lists all files including hidden files
- **Directory Navigation**
  - `cd <dir>` → Changes the current directory
  - `cd /` → Moves to the root directory
- **File Handling**
  - `touch <file>` → Creates an empty file
  - `rm <file>` → Deletes a file
- **Directory Handling**
  - `mkdir <dir>` → Creates a new directory
  - `rmdir <dir>` → Deletes an empty directory
  - `rm -r <dir>` → Deletes a directory recursively (even if not empty)
    - ✅ Always use `rm -r` for non-empty directories
- **Searching & Text Processing**
  - `find <path> -name "<file>"` → Searches for files by name in a directory hierarchy
  - `grep "<text>" <file>` → Searches for a specific text inside a file
- **File Editing**
  - `vi <file>` → Opens a file in the Vim editor for editing
- **History**

- `history` → Shows previously executed commands

Notes: Click here

Linux & Shell Scripting for DevOps

1. Introduction to Linux in DevOps

- **Why Linux?**
- Most DevOps tools (Docker, Kubernetes, Git, etc.) are optimized for Linux.
- Cloud servers (AWS, Azure, GCP) mainly run Linux.
- Stability, security, and performance.
- **Accessing Linux on Windows (WSL):**

```
wsl --install       # installs Ubuntu by default
wsl --list --verbose
wsl                 # start Ubuntu shell
```

2. Navigating File System

- **Basic Commands:**

```
pwd                 # show current directory
ls                  # list files
ls -l               # detailed list
ls -a               # include hidden files
cd <dir>            # move into directory
cd ~                # go home
cd /                # go root
```

- **File/Directory Management:**

```
touch file.txt      # create empty file
mkdir folder        # create folder
rm file.txt         # delete file
rm -r folder        # delete folder with contents
cp file1 file2      # copy file
mv file1 newloc/    # move/rename file
```

1

Shell Scripting

A **shell script** is a text file that contains a series of commands designed to be executed by the *Linux shell*.

Key aspects of shell scripting include:

- **Automation:** Its primary purpose is to automate repetitive tasks, such as system configurations, software installations, or taking backups, which significantly increases efficiency.
- **Structure:** Scripts typically start with a `shebang` line (`#!/bin/bash`), which specifies the interpreter to use.
- **Workflow:** The standard process to use a shell script involves four steps:
  1. **Create** the file (e.g., using `nano`).
  2. **Write** the commands into the file.

```
- **Save** the script and exit: **`Ctrl+'O'` -> `Enter` ->
`Ctrl+'X'`**
```

  3. **Execute** using the `chmod +x` command. (`chmod +x <file_path_from_current_directory>`)
  4. **Run** the script using `./<script_name>`.

Advanced features like **variables** and **conditional statements** can also be used to make your scripts more dynamic and powerful.

For example: conditional.sh

```
#!/bin/bash
number=6
if [ $number -gt 5 ]; then
    echo "Num greater than 5"
else
    echo "Nah, It's smaller"
fi
```

- **Commands Executed:**

```
ankit@DESKTOP-RJU5O05:/mnt/c/Users/ANKIT/Desktop$ chmod +x
conditional.sh
ankit@DESKTOP-RJU5O05:/mnt/c/Users/ANKIT/Desktop$ ./conditional.sh
Num greater than 5
```

Git & Github

- **Git:** A **distributed version control system** designed to track changes in source code locally on a machine. Because it is distributed, every developer holds a full copy of the repository and its history, eliminating the constant need for a central server.
- **GitHub:** A **cloud-based hosting service** built upon Git. It provides a platform for collaboration, access control, bug tracking, and CI/CD integration.
- **The Workflow:** The process involves tracking changes locally with Git (`add` / `commit`) and then synchronizing those changes with the remote repository on GitHub (`push` / `pull`) to ensure team members have access to the most recent code.
- **Practical Implementation**
  1. **Initialization:** Use `git init` to start tracking a project. Do not forget to use `.gitignore` file to exclude unnecessary files like `node_modules`.
  2. **Staging and Committing:**
     - `git add` : Moves changes to the staging area.
     - `git commit -m "message"` : Creates a snapshot of the current codebase. It's important to write **descriptive commit messages** to maintain clarity in team environments.
  3. **Remote Repository:**
     - `git remote add origin` : Links the local repository to the GitHub URL.
     - `git push -u origin` : Uploads local commits to GitHub.
- **Branching Strategies**
  - Branching is vital for **parallel development** and maintaining the stability of the production environment:
  - **Purpose:** Allows developers to work on new features or fixes without affecting the `main` (production) branch.
  - **Management:**
    - `git branch <branch_name>` : Create a new branch.
    - `git checkout <branch_name>` (or `git switch <branch_name>`): Switch between branches.
    - **Pull Requests (PRs):** Used to merge changes from a feature branch (e.g., `dev`) into the `main` branch. This provides a mechanism for code review and automated checks before finalizing a merge.

- **Summary of Essential Commands**
  - `git init` : Initializes repository.
  - `git status` : Checks the current state of tracked files.
  - `git add .` : Stages all changes.
  - `git commit -m "..."` : Saves a snapshot.
  - `git push origin` : Pushes changes to the cloud.
  - `git remote -v` : Verifies remote connections.

  Notes: [Click Here]

Git, GitHub & Environment Management

1. Git Basics

- **What is Git?**
- A distributed version control system (VCS).
- Tracks changes in code and enables collaboration.
- **Git Lifecycle Commands:**

```
git init             # start a repo
git status           # check changes
git add file.js      # stage file
git commit -m "msg"  # save snapshot
git log              # history
```

- **.gitignore Example:**

```
node_modules/
.env
dist/
```

2. GitHub Basics

- **What is GitHub?**
- Remote platform to host Git repositories.
- Enables collaboration, pull requests, CI/CD integrations.
- **Remote Setup:**

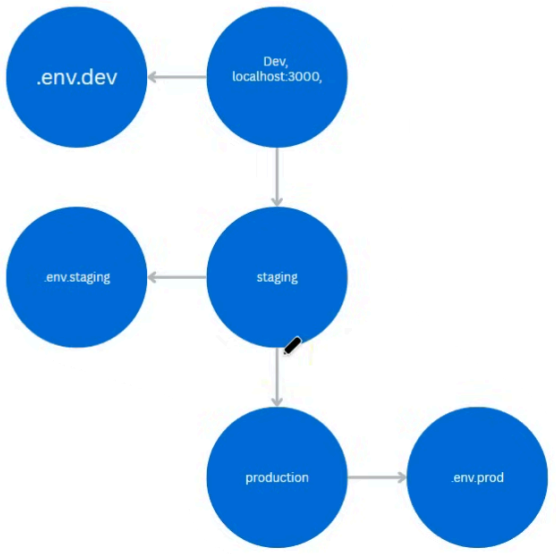
```
git remote add origin https://github.com/user/repo.git
git branch -M main
git push -u origin main
```

- **Branching Flow:**

Environment Management

It is critical for maintaining security. You should never hardcode sensitive credentials (like database URLs or API keys) directly into your source code, as these would be exposed if pushed to a public repository like GitHub.

- **The Use of .env Files**: To keep sensitive information secure, the course demonstrates using .env files. These files store environment-specific configurations locally and should be added to your .gitignore file to prevent them from being committed to version control.
- **Environment Tiers:**
  - **Development (Dev):** Used for initial coding and local testing.
  - **Staging:** A pre-production environment that mirrors the production setup. This is where you conduct final testing before deploying to live users to ensure stability.
  - **Production (Prod):** The live environment where real users access your application.



Docker

Docker is defined as a tool that allows you to package an application with everything it needs (code, libraries, runtime, and configurations) into a single **lightweight container**. This ensures the app runs consistently in any environment.

Docker Architecture: The host runs a **Docker Engine** which maintains containers. The key advantage is **isolation**; the app running inside the container is decoupled from the host OS, and you can run multiple containers side-by-side. Containers share the host OS kernel but remain logically isolated from each other.

Docker Daemon (`dockerd`): The Docker daemon is the central background service that runs on the host operating system. It acts as the "brain" of Docker, responsible for:
- **Managing Objects:** It handles the creation, distribution, and lifecycle of images, containers, networks, and volumes.
- **Client Requests:** It listens for requests from the Docker client (the command-line interface) and performs the requested tasks.
- **Streaming Output:** When you run a command like `docker run`, the daemon executes the process, manages the container environment, and streams the application output back to your terminal.

- **Key Terminology**:
  - **Image**: A "blueprint" or a read-only template (like a zip file) containing your application code and environment setup. Images are usually pulled from *Docker Hub*, which is a public image registry.
    - **Container**: The actual running instance of an image.
    - **Docker Engine**: The system that runs containers and includes the Docker daemon, which manages them.
- **Essential Commands**:
  - `docker run`: Pulls and starts a container from an image.
  - `docker run -d -p <host_port>:<container_port> <image_name>`: Starts a container in **detached mode** (-d) and maps host ports to container ports (-p).
  - `docker ps -a`: Lists all containers.
  - `docker stop`: Stops a running container.
  - `docker rm`: Removes a container.
  - **Building/Running**: Using `docker build -t <image_name> .` to tag the image and `docker run` to execute it.

```
C:\Users\sanga>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:a0dfb02aac212703bfcb339d77d47ec32c8706ff250850ecc0e19c8737b18567
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

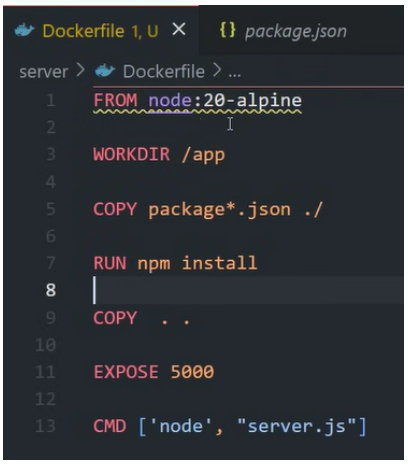
Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

A `Dockerfile` is a text document containing all the instructions needed to build a container image. Here is a breakdown of the essential commands used:

- `FROM` : Specifies the base image to start your build. For example, `node:20-alpine` uses a lightweight Linux distribution with Node.js pre-installed.
- `WORKDIR` : Sets the working directory inside the container. All subsequent commands (like `COPY` or `RUN`) are executed relative to this path. It essentially creates a folder (e.g., `/app`) inside the container's virtual filesystem.
- `COPY` : Transfers files from your local host machine into the container's filesystem.
  - `COPY package.json .` copies the manifest file to the current `WORKDIR` .
  - `COPY . .` copies everything from the current local directory → into the current working directory inside the container.
- `RUN` : Executes commands during the image build process (e.g., `npm install` to download dependencies).
  - Copying `package.json` first and running `npm install` helps in caching dependencies. This avoids reinstalling dependencies every time code changes, making builds faster.
- `CMD` : Defines the default command that executes when a container is **started**. Unlike `RUN` , which executes during the build, `CMD` runs only when the container is actually launched.
- `EXPOSE` : Specifies which port the container listens on (e.g., `EXPOSE 5000`). It is mainly for documentation; actual port mapping happens using `-p` in

`docker run`.



Docker Compose

Why Docker Compose is Needed:

- **Complexity Management:** Manually building and running separate containers for each service is time-consuming and prone to errors.
- **Orchestration:** Think of Docker CLI as a single musician, while *Docker Compose* is an entire orchestra. It allows for defining and running multi-container applications in a single YAML file (`docker-compose.yml`), handling services, networking, and volumes centrally.

Key Concepts and Structure:

- **The YAML Configuration:** You define your **services** (e.g., `back-end`, `front-end`) within the file. Each service specifies its **build** context, **container name**, **environment files**, and **port mappings**.
- **Service Interdependency:** You can use the `depends_on` instruction to define the order in which services should start, ensuring the backend is ready before the frontend.
- **Automatic Networking:** Containers defined in the same Compose file are automatically connected through a shared, private network.

Essential Docker Compose Commands:

1. `docker compose up --build`: Starts all defined services and triggers a build process for the containers.
2. `docker compose up -d --build`: Same as above, but runs the containers in **detached mode** (in the background).

3. `docker compose down` : Stops and removes all containers and networks associated with the project.
4. `docker compose logs` : Displays logs for all services. To see logs for a specific service, use `docker compose logs <service_name>` (e.g., `docker compose logs back-end`).

Question: Why are individual *Dockerfiles* still required if you are already using *Docker Compose* to manage your services?

- **Dockerfiles** are the fundamental blueprints that define **how to build** a specific application image (e.g., installing dependencies, copying code, setting environments).
- **Docker Compose** acts as an **orchestrator**; its role is not to build the images from scratch but to manage the lifecycle of multiple containers simultaneously (defining services, networking, and dependencies).
- When *Docker Compose* performs a build, it actually references the individual *Dockerfiles* for each service to create the necessary images.
  Notes: Click Here

Docker & Docker Compose

1. Docker Basics

- **What is Docker?**
- Packages app + dependencies into containers.
- Ensures app runs the same on all systems.
- **Why Docker?**

Continuous Integration (CI)

At its core, **CI** is a development practice where multiple contributors automatically integrate their code changes into a shared codebase. Rather than merging code manually and facing integration conflicts, CI automates the process of verifying that new code doesn't break the existing application.

Key benefits of CI include:

- **Automated Feedback:** You receive immediate feedback on whether your code passes linting, building, and testing.
- **Team Collaboration:** It allows multiple developers to push to a shared repository safely without disrupting the `master/main` branch.
- **Deployment-Ready Code:** Because the code is constantly being built and verified, the main branch remains in a stable, deployment-ready state.

The CI Workflow
To maintain a stable production environment, the process typically follows these rules:

1. **Branching Strategy:** Developers do not push directly to the `master` or `main` branch. Instead, they use **feature branches**.
2. **Pull Requests (PRs):** Once a feature is complete, a developer raises a PR to merge into the `dev` or `main` branch.
3. **Automated Checks:** Triggered by the PR, the CI workflow runs checks. If any part of the process (linting, testing, or building) fails, the PR is blocked, preventing the merge.

Practical Implementation with GitHub Actions
The course demonstrates how to implement this using **GitHub Actions**, a powerful automation tool built directly into GitHub.

- **Setup:** You create a directory structure at the root of your project: `.github/workflows/`.
- **Workflow Files:** Inside this folder, you place a `yml` file (e.g., `ci.yml`). Think of this as a script that tells GitHub exactly what to run every time code is pushed.
- **Linting & Prettier:** You configure **ESLint** and **Prettier** for both the client and server directories. These tools ensure your code follows a consistent style and catch potential syntax errors automatically during the CI build process.

Summary of the Flow

1. **Code Change:** A developer pushes code to a feature branch.
2. **Workflow Trigger:** GitHub Actions detects the push and starts the defined workflow.
3. **Job Execution:**
   - It pulls the repository.
   - It installs dependencies.
   - It runs linting checks.
   - It executes build commands.
4. **Result:** If everything passes, the PR is marked as safe to merge; if it fails, the developer is alerted to fix the issues immediately.

Github Actions, ESlint and Prettier

Whenever code is pushed, the GitHub Action automatically triggers to run the tests (defined in `.ithub/workflows`). If the build or linting fails (e.g., due to an unused variable), the pipeline reports an error, providing immediate feedback to the developer.

Setting Up Branch Protection:

- To enforce the standards defined in the CI pipeline, configure the repository to act as a 'gatekeeper' before any code is merged:

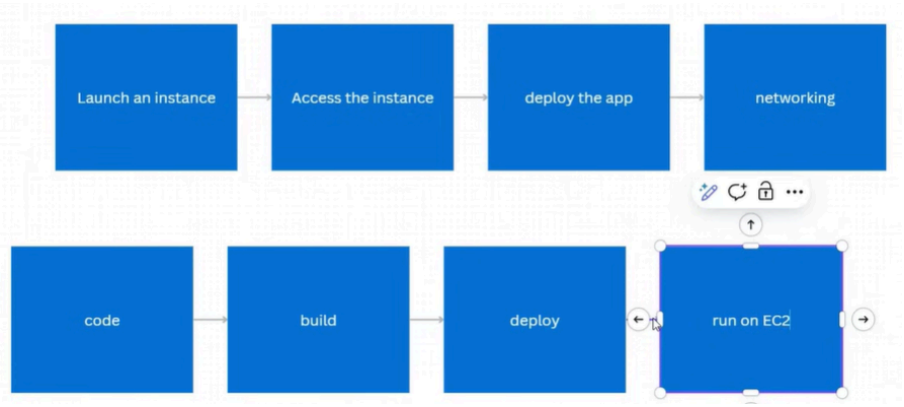
- **Accessing Settings:** Navigate to the repository **Settings** tab and selects **Branches** to define rules for the *main* branch.
- **Requiring Status Checks:** Enable the **Require status checks to pass before merging** option. This ensures that the GitHub Action (which performs linting and builds) must report a 'success' status before the merge button becomes active.
- **Blocking Insecure Merges:** By requiring these checks, it prevents developers from bypassing the linting process. If a developer attempts to merge code that fails the ESlint or Prettier tests, the UI explicitly blocks the merge.

  Notes: Click Here

Continuous Integration (CI) with ESLint & Prettier

AWS EC2 Deployment

1. Understanding AWS EC2

- **EC2 Defined:** *Elastic Compute Cloud* (EC2) is a core *AWS* service that allows users to rent virtual servers called **instances**.
- **Core Concepts:** There are several essential components users must understand:
  - **AMI (Amazon Machine Image):** Pre-configured OS templates.
  - **Security Groups:** Used as a firewall to manage network traffic and open necessary ports (like 80 or 3000) for web access.
  - **Key Pairs:** Critical for secure *SSH* connectivity, allowing the user to log into the remote instance from a local terminal.
- 

2. Setup and SSH Access

- **Connectivity:** Use the `.pem` file generated by AWS to establish a secure *SSH* connection from a local terminal to the instance's public IP.

3. Server Configuration & Environment Preparation

Once inside the instance, execute a series of scripted steps to prepare the environment:

- **Environment Setup:** Installing necessary packages including *Docker*, *Docker Compose*, and *Git*.
- **Repository Management:** Cloning the project code directly onto

the *EC2* instance.

- **Environment Variables:** He manually creates `.env` files for both the *server* and *client* directories to handle production configurations, emphasizing that sensitive data should not be hardcoded.

4. Docker Integration & Networking

- **Dockerignore:** To optimize the build, the instructor introduces the `.dockerignore` file. Much like `.gitignore`, this file ensures that unnecessary files (like `node_modules` or local `.env` files) are excluded from the *Docker* build context, keeping images small and secure.
- **Networking & Ports:** A crucial step is configuring the application to bind to `0.0.0.0` instead of `localhost` inside the containers. This ensures the application is accessible from the internet through the *EC2* instance's public IP address.

5. Final Deployment

- **Execution:** The process finishes by running `docker compose up -d` to launch the containers in detached mode.
- **Verification:** Access the public IP of the *EC2* instance in a browser to confirm the `application` is fully functional and live on the web.
  Notes: Click Here

AWS EC2 Deployment

---

1. What is AWS EC2?

- **Elastic Compute Cloud (EC2):** Virtual servers on AWS.
- **Elastic = scalable** → increase/decrease resources on demand.
- **Use in DevOps:** Run backend, frontend, and DB in the cloud.

---

2. AWS Account Setup

1. Go to aws.amazon.com.
2. Click **Create an AWS Account**.
3. Provide email, password, credit card (for verification).
4. Choose **Free Tier** → eligible for **t2.micro** instance.

Continuous Deployment (CD)

Definition: CD is a DevOps practice where code changes are automatically released to production after passing all **CI (Continuous Integration)** checks.
The Goal: To remove the manual process of pulling the latest code and restarting containers every time a developer merges changes.

1. Workflow Automation Logic

To automate the deployment, the instructor configures the *GitHub Actions* YAML file with specific logic:

- **Job Dependencies (needs):** He ensures the deployment job only runs if the **CI (build and lint) job** has successfully completed. This prevents broken code from ever reaching the server.
- **Event Triggers:** The pipeline is configured to trigger specifically on **push** events to the repository.

2. Secure Server Connectivity and GitHub Secrets

Because the deployment runs automatically, the *GitHub* runner needs secure, non-interactive access to the *EC2* instance. So, we add four specific **GitHub Secrets** in the repository settings (**Settings > Secrets and variables > Actions**), hence, avoiding hardcoding sensitive data:

- **EC2_HOST:** The public IP address of your *AWS EC2* instance.

- **EC2_USER:** The username used to log into the server (e.g., *ec2-user*).
- **EC2_SSH_KEY:** The content of your private `.pem` key file for authentication.
- **APP_DIR:** The destination path on the server where the application code resides.

SSH Security in Workflow:

- **SSH Fingerprinting:** The workflow includes a command to fetch the server's SSH fingerprint and add it to *known_hosts*. This bypasses the manual "Are you sure you want to continue connecting?" prompt that would otherwise block automated scripts.
- **Private Key Injection:** He demonstrates how the runner uses the stored *EC2_SSH_KEY* to establish a secure, authorized connection to the server (Refer Notes).

3. Real-world Demonstration

- **The Process:** After configuring the workflow, the instructor performs a live change: he updates the application code and pushes it. The *GitHub Action* triggers the build/lint process, followed by the automatic deployment on the *EC2* server.
- **Verification:** He refreshes the browser page pointing to the *EC2* public IP, showing that the changes appear instantly without any further manual action on the server.
  Notes: Click Here

Continuous Delivery (CD) with GitHub Actions & EC2

---

1. What is CD?

- **Continuous Delivery:** Automating deployment after CI.
- Push code → build/lint (CI) → deploy to server (CD).
- Ensures latest version is always running on EC2.

---

2. GitHub Secrets Setup

In your repo → **Settings → Secrets and variables → Actions → New secret**

Add: - `EC2_HOST` → `<EC2_PUBLIC_IP>` - `EC2_USER` → `ec2-user` - `EC2_KEY` → contents of `.pem` file - `EC2_APP_DIR` → `/home/ec2-user/mern-app`

---

3. CD Workflow File

Kubernetes

Kubernetes (also written as **K8s** — because there are 8 letters between K and s) is an **open-source container orchestration system**, originally built by Google, now maintained by the Cloud Native Computing Foundation (CNCF).
In plain terms: **it is a tool that manages your containers for you**, deciding where they run, keeping them alive, scaling them up/down, and letting them talk to each other.

Why Kubernetes? (The Problem First)

- You already know Docker. A container packages your app and its dependencies. That's great for running *one thing* on *one machine*. But real production apps have problems Docker alone can't solve. Running containers with just EC2 + Docker Compose don't solve it either:

Problem	What Happens Without K8s
Backend crashes	Site goes down until you SSH in and restart manually
Traffic spikes (1k → 50k users)	No auto-scaling, server just dies under load

Problem	What Happens Without K8s
Frontend needs backend address	You hardcode IP/PORT, which breaks whenever container restarts
You have 10 microservices	You're manually managing 10 containers across multiple machines

- Kubernetes was built specifically to solve all of these.
- "**The Declarative Model**" is a core philosophy of K8s. Instead of writing step-by-step commands like:

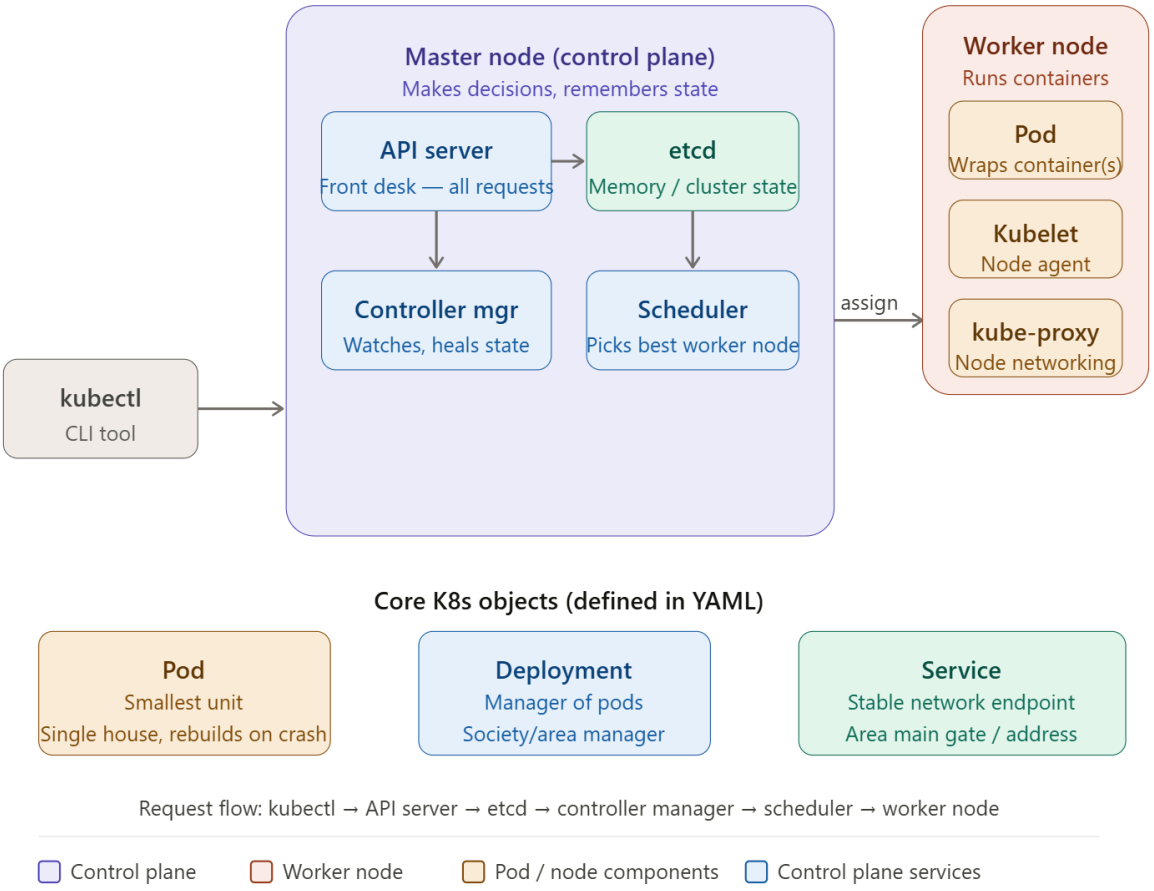
  > "Start 3 containers, then if one dies, start another, then route traffic to all three..."

- We write a YAML file that says:

  > "I want 3 replicas of this container always running."

- Kubernetes reads that and figures out *how* to make it happen — and keeps checking forever to make sure reality matches what you declared. This is called **declarative infrastructure**.
- **Key benefits:** Auto-scaling, self-healing (auto-restarts crashed pods), load balancing, and portability (run anywhere: cloud, local, on-prem).

The Two Halves of a K8s Cluster



- **Master Node (Control Plane) — "The Brain"**
  - This is the decision-making part. It never runs your actual app, it just manages everything.
  - **API Server**: the single entry point for all requests. When you run any `kubectl` command, it hits the API Server first. Think of it as the *front desk* of a company.
  - **etcd**: a key-value store that holds the entire cluster's state. Every pod, every service, every deployment config is stored here. If the API Server is the front desk, etcd is the *memory/database* behind it.
  - **Controller Manager**: watches the current state and compares it to the desired state. If a pod crashes, the controller manager notices and tells the scheduler to fix it. This is what gives K8s its self-healing property.
  - **Scheduler**: acts like a *dispatcher*. Once it knows a pod needs to be placed, it picks the best available worker node based on resources (CPU, memory) and places the pod there.
    - In K8s, a **node is simply a server**, it's a machine (real or virtual) that has CPU, RAM, and storage. That's it. It could be:

        - An EC2 instance on AWS
        - A VM on your laptop (like what Minikube creates)
- **Worker Node — "Where Your App Actually Runs"**
    - **Pod**: the smallest deployable unit. It wraps one (or sometimes more) container. My Node.js backend runs inside a pod. Think of it as a *single house*. If it breaks, K8s rebuilds a new one automatically.
    - **Kubelet**: an agent running on every worker node. It talks to the master node and ensures the containers in pods are running as instructed.
    - **kube-proxy**: handles node-level networking. It provides simple load balancing so that traffic reaching the node gets routed to the right pod.
- **Core K8s Objects (you define these in YAML)**
    - **Pod**: as above, the single house. We rarely create pods directly; we let a Deployment manage them.
    - **Deployment**: the *manager of pods*. We tell it "I want 3 replicas of my backend pod." It creates them, keeps them alive, and handles rolling updates. Think of it as a *society manager* or *area manager* overseeing a colony of identical houses.
    - **Service**: provides a stable address (IP + port) to access a set of pods. Since pods can die and be recreated with new IPs, a Service acts as the *main gate with a fixed address* — our frontend always talks to the Service, not directly to pods. This is also how the backend finds the database.
- **The Full Request Flow**

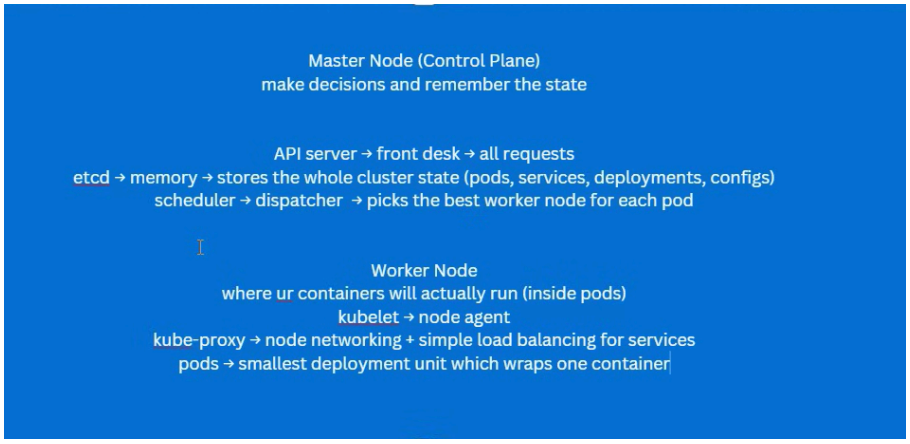
```
You (kubectl command)
  → API Server          [receives request, validates it]
  → etcd                [stores the desired state]
  → Controller Manager  [detects gap between desired vs actual]
  → Scheduler           [picks the best worker node]
  → Worker Node         [kubelet creates the pod, app runs]
```

- **Practical Tools**
    - **kubectl**: the command-line tool we use to interact with the cluster (create deployments, check pod status, view logs, etc.). A **Kubernetes cluster** is just a collection of nodes (machines) working together:
        - ```
          Cluster
          ├── Master Node  →  1 machine acting as the brain
          ```
          ```

```
(control plane)
├── Worker Node  →  1 machine running your apps
├── Worker Node  →  another machine running your apps
└── Worker Node  →  another machine running your apps

Node (the machine)
    └── Pod (runs inside the machine)
        └── Container (runs inside the pod)
```

- **Minikube**: Here everything runs on **one single node**, our laptop acts as both master and worker (for local learning).



Docker = packages our app into a container (solves "it works on my machine").
Kubernetes = manages those containers at scale across many machines (solves "how do I run this reliably in production").
Notes: Click Here

Kubernetes Basics with MERN Example

---

1. What is Kubernetes?

- **Container Orchestration Platform** → manages Docker containers.
- Handles deployment, scaling, networking, and self-healing.
- Works across clusters of machines.

Why Needed? - Docker runs containers, but scaling & reliability = manual. - Kubernetes automates scaling, restarts crashed containers, and provides stable networking.

---

2. Kubernetes Architecture (Simple)

- **Control Plane (Master):** API Server, Scheduler, Controller Manager, etcd.
- **Worker Nodes:** Run actual Pods (containers).
- **kubectl:** CLI to interact with cluster.

Analogy: - Control Plane = Air traffic control tower ✈️ - Worker Nodes = Runways - Pods = Planes (carry containers)